# Week 2 - Day 3 (Wednesday): State Machine and Telemetry Packet Builder

**Intern:** Likhitha N | **Role:** IoT / Embedded | **Company:** CRASH CONNECT PVT LTD

> **SIMULATED / TEST DATA.** GPS is recorded NMEA data, IMU, SOS and network are test inputs. Nothing here comes from real hardware.

## 1. Device-state structure

The state machine is in `src/state/`. One function, `state_next(current, inputs)`, takes the current state and a set of inputs and returns the next state and the reason. Keeping it as one function makes it easy to test.

| Input | Meaning | Comes from |
|---|---|---|
| gps_fix | A valid, recent GNSS fix exists | gnss |
| network_up | Cellular link is connected | comms |
| event | NONE, IMPACT_SUSPECTED or SOS | safety_event |
| critical_fault | A critical fault flag is set | fault |
| noncritical_fault | A non-critical fault flag is set (for example IMU not responding) | fault |
| self_test_pass | Result of the self-test run when SELF_TEST was entered | main |
| time_in_state | How long the device has been in the current state | state |

## 2. Transitions implemented

| From | Condition | To |
|---|---|---|
| any state except FAULT | critical fault | FAULT |
| BOOT | boot time over | SELF_TEST |
| SELF_TEST | self-test time over and passed | ACQUIRING_GPS |
| SELF_TEST | self-test time over and failed | FAULT |
| ACQUIRING_GPS | GPS fix | NETWORK_CONNECTING |
| ACQUIRING_GPS | GPS timeout | DEGRADED |
| NETWORK_CONNECTING | network up and GPS fix | OPERATIONAL |
| NETWORK_CONNECTING | network up but no GPS fix, or timeout | DEGRADED |
| OPERATIONAL | event | EVENT_ACTIVE |
| OPERATIONAL | GPS fix lost, network lost or non-critical fault | DEGRADED |
| EVENT_ACTIVE | event gone and hold time over, system good | OPERATIONAL |
| EVENT_ACTIVE | event gone and hold time over, system degraded | DEGRADED |
| DEGRADED | event | EVENT_ACTIVE |
| DEGRADED | GPS fix, network up and no fault | OPERATIONAL |
| FAULT | fault cleared and restart delay over | BOOT |

## 3. Placeholder test inputs and the demonstration

The `sim` module changes the inputs on a timetable so the transitions can be shown without hardware. Each change is printed as `[SIM][TEST INPUT]`.

| Time | Test input | Result |
|---|---|---|
| 0 s | No GPS signal, network down, IMU stationary | BOOT, SELF_TEST, ACQUIRING_GPS |
| 4 s | Corrupted NMEA sentence | Rejected by checksum |
| 5 s | GPS signal available | NETWORK_CONNECTING |
| 9 s | Network available | OPERATIONAL |
| 16 s, 19 s | IMU tilt, then movement | No event (below threshold) |
| 22 s | IMU impact test values | EVENT_ACTIVE (faster telemetry) |
| 24 s | IMU stationary | Back to OPERATIONAL after the hold time |
| 34 s | GPS signal lost | DEGRADED, packets carry null location |
| 40 s | GPS signal back | OPERATIONAL |
| 46 s | Network lost | DEGRADED, a packet is not sent |
| 53 s | Network back | OPERATIONAL |
| 56 s, 58 s | SOS pressed, released | EVENT_ACTIVE, then OPERATIONAL |
| 66 s | Critical fault injected | FAULT |
| 70 s | Fault cleared | BOOT, SELF_TEST, ACQUIRING_GPS, NETWORK_CONNECTING, OPERATIONAL |

State changes printed by the demonstration:

```
[STATE] t=2.0s BOOT -> SELF_TEST (boot complete)
[STATE] t=3.0s SELF_TEST -> ACQUIRING_GPS (self-test passed)
[STATE] t=6.0s ACQUIRING_GPS -> NETWORK_CONNECTING (GPS fix
acquired)
[STATE] t=10.0s NETWORK_CONNECTING -> OPERATIONAL (network
connected)
[STATE] t=23.0s OPERATIONAL -> EVENT_ACTIVE (event detected)
[STATE] t=27.0s EVENT_ACTIVE -> OPERATIONAL (event cleared)
[STATE] t=35.0s OPERATIONAL -> DEGRADED (GPS fix lost)
[STATE] t=41.0s DEGRADED -> OPERATIONAL (recovered)
[STATE] t=47.0s OPERATIONAL -> DEGRADED (network lost)
[STATE] t=54.0s DEGRADED -> OPERATIONAL (recovered)
[STATE] t=57.0s OPERATIONAL -> EVENT_ACTIVE (event detected)
[STATE] t=61.0s EVENT_ACTIVE -> OPERATIONAL (event cleared)
[STATE] t=67.0s OPERATIONAL -> FAULT (critical fault)
[STATE] t=71.0s FAULT -> BOOT (fault cleared, restarting)
[STATE] t=72.0s BOOT -> SELF_TEST (boot complete)
[STATE] t=73.0s SELF_TEST -> ACQUIRING_GPS (self-test passed)
[STATE] t=73.1s ACQUIRING_GPS -> NETWORK_CONNECTING (GPS fix
acquired)
[STATE] t=73.2s NETWORK_CONNECTING -> OPERATIONAL (network
connected)
```

## 4. Telemetry packet builder

`src/telemetry/` builds one JSON packet from the current readings. **The common packet fields were to be agreed in the joint session. I did not have the agreed list when I wrote this, so the field names below are my draft (packet version `0.1-DRAFT`). They will be replaced by the agreed fields (Pending Confirmation).**

| Field | Type | Source | When unavailable |
|---|---|---|---|
| packet_version | text | config | always present |
| device_id | text | config | always present |
| seq | number | counter | always present |
| uptime_ms | number | main | always present |
| data_source | SIMULATED or HARDWARE | config | always present |

| | | | |
|---|---|---|---|
| device_state | text | state | always present |
| timestamp_utc | text | gnss | null without a valid fix |
| location.fix | true / false | gnss | false |
| location.latitude, longitude | number | gnss | null |
| location.altitude_m, speed_kmh, hdop | number | gnss | null |
| location.satellites | number | gnss | null (a real value of 0 is kept as 0) |
| motion.accel_x/y/z_g | number | imu | null |
| motion.gyro_x/y/z_dps | number | imu | null |
| motion.accel_magnitude_g | number | imu | null |
| event.type | NONE, IMPACT_SUSPECTED, SOS | safety_event | always present |
| event.sos_active | true / false | sos | always present |
| comms.connected | true / false | comms | always present |
| battery_mv | number | power | null (stub) |
| fault_flags | number | fault | always present |

**Handling unavailable values**

- Unavailable values are sent as `null`, never as `0`. A latitude and longitude of 0, 0 is a real place, so 0 would be misleading.
- Location fields are only filled when the fix is valid and recent. The old position is never reused after a "no fix" sentence.
- A real zero (for example 0 satellites) stays 0 and is only null when the value was never received.
- If the packet does not fit in the buffer, the builder returns 0 and sends nothing instead of sending a cut-off packet.

**Example packet with a valid fix (OPERATIONAL)**

```
{
  "packet_version": "0.1-DRAFT",
  "device_id": "CRASH_ESP32_01",
  "seq": 1,
  "uptime_ms": 10000,
  "data_source": "SIMULATED",
  "device_state": "OPERATIONAL",
  "timestamp_utc": "1994-03-23T12:35:19Z",
  "location": {
    "fix": true,
    "latitude": 48.1173,
    "longitude": 11.516667,
    "altitude_m": 545.4,
    "speed_kmh": 41.5,
    "satellites": 8,
    "hdop": 0.9
  },
  "motion": {
    "accel_x_g": 0.02,
    "accel_y_g": 0.0,
    "accel_z_g": 1.0,
    "gyro_x_dps": 0.6,
    "gyro_y_dps": 0.1,
    "gyro_z_dps": 1.1,
    "accel_magnitude_g": 1.0
  },
  "event": {
    "type": "NONE",
    "sos_active": false
  },
  "comms": {
    "connected": true
  },
  "battery_mv": null,
  "fault_flags": 0
}
```


**Example packet without a fix (DEGRADED)**


```
{
  "packet_version": "0.1-DRAFT",
  "device_id": "CRASH_ESP32_01",
  "seq": 9,
  "uptime_ms": 36000,
  "data_source": "SIMULATED",
  "device_state": "DEGRADED",
  "timestamp_utc": null,
  "location": {
    "fix": false,
    "latitude": null,
    "longitude": null,
    "altitude_m": null,
    "speed_kmh": null,
    "satellites": 0,
    "hdop": null
  },
  "motion": {
    "accel_x_g": 0.02,
    "accel_y_g": 0.0,
    "accel_z_g": 1.0,
    "gyro_x_dps": 0.6,
    "gyro_y_dps": 0.1,
    "gyro_z_dps": 1.1,
    "accel_magnitude_g": 1.0
  },
  "event": {
    "type": "NONE",
    "sos_active": false
  },
  "comms": {
    "connected": true
  },
  "battery_mv": null,
  "fault_flags": 0
}
```

## 5. Test results

- 66 unit tests passed, 0 failed. They cover every state transition, the fault flags, the event rule, GNSS parsing (GP and GN talkers, no fix, wrong checksum, stale data) and the packet builder (all values, unavailable values, buffer too small).
- The demonstration ran the full timetable above. 20 packets were "sent" to the stub, 1 was not sent because the network was down, and 1 corrupted sentence was rejected.
- All 25 packets printed by the tests and the demonstration are valid JSON.

Full output: `test_results/unit_test_output.txt` and `test_results/scenario_output.txt`.

## 6. What this does not prove

- It ran on a PC with a stand-in for the Arduino functions, not on an ESP32-S3.
- No real GNSS, IMU, button or LTE module was used.
- The impact rule (acceleration magnitude at or above 4.0 g) is only a **placeholder to demonstrate the state change**. It is not accident detection.

## 7. Pending Confirmation (Day 3)

- The agreed common packet fields, names, units and format.
- Event message format (whether it is a separate message or the same packet with an event field).
- Timestamp when there is no GNSS fix (null for now).
- What happens to packets when the network is down (they are dropped for now, no buffering).
- Whether telemetry should be sent in FAULT, and the restart policy after a fault.
- Whether SOS must work before the first GPS fix and while connecting to the network.